

# Proof-Carrying-BPF Programs for the eBPF Verifier

Yalu Cai  
Rutgers University  
Piscataway, USA  
yc1795@cs.rutgers.edu

Santosh Nagarakatte  
Rutgers University  
Piscataway, USA  
santosh.nagarakatte@cs.rutgers.edu

## Abstract

This paper proposes proof-carrying-BPF programs as a mechanism to increase the class of programs accepted by the Linux eBPF verifier without increasing its complexity. To accomplish this vision, we develop a userspace version of the kernel verifier and append it to the BPF backend. The BPF backend generates proofs with symbolic evaluation where the kernel verifier is not able to prove safety. The kernel verifier checks proofs with an in-kernel proof checker and accepts the program when the proof is valid. This coupling of the userspace BPF verifier to the BPF backend addresses the limitations of the recent BCF framework, such as latency due to multiple kernel-to-userspace transitions and the dormant verifier state during userspace verification. Our prototype successfully verifies all 360 objects of the BCF suite – drawn from the Calico and Cilium datapaths, BCC, and Inspektor Gadget – that the kernel verifier previously rejected.

**CCS Concepts:** • Software and its engineering → Formal methods; • Theory of computation → Program reasoning.

**Keywords:** eBPF verifier, proof-carrying BPF

## ACM Reference Format:

Yalu Cai and Santosh Nagarakatte. 2026. Proof-Carrying-BPF Programs for the eBPF Verifier. In *4th Workshop on eBPF and Kernel Extensions (eBPF'26)*, September 29–October 02, 2026, Prague, Czech Republic. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3837779.3838158>

## 1 Introduction

The eBPF ecosystem is widely used to extend the functionality of the Linux kernel at runtime, especially in cloud-native environments for networking and for security. Developers can attach eBPF programs at various points in the kernel at runtime. These programs are checked by the “eBPF verifier”, which is a static analyzer in the kernel, when such programs are loaded. Once the verifier determines that the program is

safe for all inputs (e.g., memory safety and bounded control-flow), the kernel just-in-time compiles the program to the target ISA and attaches it to the appropriate hooks.

**Analysis by the eBPF verifier.** The verifier symbolically explores every possible path in the eBPF program and accepts a program when all paths are safe for all inputs. The verifier uses simple non-relational abstract domains such as the intervals (32- and 64-bit signed/unsigned intervals) and tristate numbers to perform its static analysis. Any soundness bug in the verifier (i.e., accepts an invalid program) can result in arbitrary kernel accesses and privilege escalation attacks [11, 12, 18, 19]. Hence, there have been numerous efforts to test or formally verify the soundness of various aspects of the eBPF verifier [2, 8, 9, 13, 22, 24–26, 28, 29].

**Fight with the verifier.** Typically, any static analysis overapproximates the set of possible behaviors and will reject some valid programs. The non-relational domains used by the kernel verifier increase the likelihood that programs will be rejected due to overapproximation (i.e., imprecision) [25]. Further, most developers use high-level languages such as C or Rust to write BPF programs. Compiling the same program under a different optimization level can cause the resulting code to be rejected by the verifier. Developers are often frustrated by seemingly valid programs being rejected, which is often referred to as “fighting the verifier”.

**Increasing the class of programs accepted.** There have been numerous efforts to improve the precision of the abstract domains used by the verifier [3, 5, 14, 21, 27, 28, 32], synthesis of precise operators [30], and precision testing [23]. Although these efforts improve the precision of existing abstract domains, they are limited by the inherent limitations of non-relational domains such as intervals and tnums. One option is to explore expressive abstract domains such as Zones or Octagons [6], which will increase the complexity of the verifier code in the kernel. Another option is to move verification to user space, similar to the PREVAIL [7] analyzer in the eBPF for Windows ecosystem. However, it requires new infrastructure for trusting externally verified code in the kernel, which may not come to fruition. Our research goal is to increase the expressivity of the programs accepted by the kernel without increasing its complexity.

**The BCF framework.** The recent BCF project [25] also aims to increase the class of programs accepted by adding a refinement procedure when the kernel verifier would reject the program. In the BCF framework, when the kernel verifier encounters a check that it cannot prove safe (e.g., `r1 = *(u64`



This work is licensed under a Creative Commons Attribution 4.0 International License.

*eBPF'26, Prague, Czech Republic*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2911-9/2026/09

<https://doi.org/10.1145/3837779.3838158>

$\ast)(r10 - 8))$ , the kernel creates a symbolic expression for all registers relevant to the instruction (*i.e.*,  $r10$  and stack location at address  $r10 - 8$ ) along with the path condition for the path being verified, which is the conjunction of the branch decisions taken to reach the program point. The kernel saves the verification state and discharges the condition to userspace, where an untrusted SMT solver attempts to prove its negation. If the solver returns UNSAT, then the condition that caused the rejection can never occur. The solver also emits a proof. The proof is then returned to the kernel via a second bpf syscall. The kernel verifier will check the proof with a small in-kernel proof checker. When the check succeeds, the kernel will resume the verification from where it left off. The paper on the BCF framework demonstrates that BCF can increase the class of programs accepted [25]. Although BCF is appealing, it requires multiple save/restores of the verifier state on verification failures, which increases latency and adds unnecessary complexity to suspend verifier execution and save/restore state.

**Proof-carrying BPF.** Inspired by proof-carrying code [16] and the BCF framework [25], we propose the **Proof-Carrying-BPF (PCB)** framework that produces BPF programs equipped with proofs where the kernel’s default verifier would reject such a program. We create a userspace version of the kernel verifier and couple it with the BPF backend. When the userspace verifier identifies program points where the kernel would reject, it explores richer symbolic analysis with an SMT solver. If it can prove that the access is safe, then it produces a proof, which can be checked by the kernel checker. Coupling the userspace verifier with the BPF backend ensures that any object code generated for the safe program will pass the kernel verifier, which avoids the frustration that BPF developers often face. Our BPF backend coupled with the userspace verifier analyzes the entire program and aggregates the proofs at all the program points where the kernel can potentially reject the program with its default analysis. This eliminates BCF’s suspensions of the in-kernel verifier: the kernel neither saves and restores verifier state nor transitions to userspace while verifying a program.

We validate the fidelity of our framework using the kernel’s selftests. It matches the kernel’s result on all 3,267 tests in the suite (zero false accepts and zero false rejects). Our PCB framework allows the kernel to accept all 360 programs from the Calico and Cilium CNI datapaths, BCC, and the Inspektor Gadget seccomp tools, which were previously rejected in the absence of proofs. PCB’s ability to precompute the proofs makes loading a BPF program 3.4 $\times$  faster than BCF on average (and faster on every object). Our prototype is open source and publicly available [4].

## 2 Background on BCF

We provide background on the BCF framework [25] because we use its in-kernel checker and extend its mechanisms.

### PC: Instruction

```

P0: R0 = call helper() ; R0 = some unknown value
P1: R0 &= 15 ; R0 ∈ [0,15]
P2: R1 = R10 ; stack frame pointer
P3: R1 += -16 ; R1 = base of a 16-byte stack buffer
P4: R1 += R0 ; R1 = base + R0, stack pointer
P5: R2 = 15
P6: R2 -= R0 ; R2 = 15 - R0 ∈ [0,15]
P7: if R2 < 4 goto P11
    ; -- fall-through (R2 ≥ 4, i.e. R0 ≤ 11) --
P8: R1 += 4 ; R1 = base + R0 + 4
P9: R0 = *(u8*)(R1 + 0) ; READ #1, rejected by kernel
P10: exit
    ; -- branch taken (R2 < 4, i.e. R0 ≥ 12) --
P11: R1 += R2 ; R1 = base + R0 + (15 - R0) = base + 15
P12: R0 = *(u8*)(R1 + 0) ; READ #2, also rejected by kernel
P13: R0 = 0
P14: exit

```

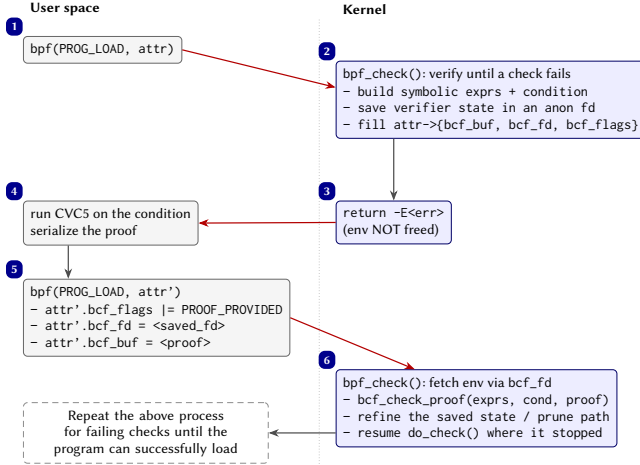
**Figure 1.** An example BPF program in the BPF ISA that is rejected by the default eBPF verifier in the Linux kernel. There are two reads (at P9 and P12) where the kernel verifier will reject the program.

**Example.** We illustrate the mechanisms in BCF and its limitations using the example program in Figure 1. In this example, there is a buffer on the stack that is of size 16 bytes (*i.e.*, valid accesses are of the form  $base + off$  where  $off \in [0, 15]$ ). If we perform full symbolic evaluation, we can deduce that both reads are safe. However, the kernel verifier tracks registers  $R0$  and  $R2$  as independent intervals. It deduces that  $R1$ ’s interval will be  $base + [4, 19]$  at P9, which is out-of-bounds. Similarly, it deduces  $R1$ ’s interval will be  $base + [0, 18]$  at P12, which is also out-of-bounds. The kernel verifier loses the relation  $R2 = 15 - R0$  and rejects the program at reads performed at program points P9 and P12.

The BCF framework extends the kernel verifier with a refinement procedure when the default verification fails. It adds mechanisms to check proofs with an in-kernel proof checker, generate symbolic expressions, and save the verifier state to suspend/resume verification in the kernel. It also extends the CVC5 SMT solver to generate proofs that will be passed to the in-kernel checker.

**BCF in action.** When the kernel encounters a check that fails with its default analysis, it triggers the refinement procedure. This process of refinement is illustrated in Figure 2. The kernel performs a backwards analysis to identify a directed acyclic graph (DAG) of all expressions that influence the registers (or memory locations) that it cannot prove safe. Once the DAG is identified, it assigns a fresh symbolic value to each register or stack location that is a leaf in the DAG. Subsequently, the kernel performs a forward analysis to build symbolic expressions along with the path condition to create a refinement condition, which encodes the condition for the access that triggered the refinement procedure to be unsafe (see Step 2 in Figure 2).

**Refinement condition for the example.** For the example program in Figure 1, the first check at P9 fails. Hence, the kernel performs the refinement procedure. The read at P9



**Figure 2.** The userspace to kernel transitions with the BCF framework. For every verification failure, there is a transition to userspace and the verifier state is saved in an anonymous file descriptor. Once the userspace solves the symbolic expression with the CVC5 solver, it sends the proof with another bpf syscall. The transitions from userspace to kernel and back are shown in red.

reads register R1. The kernel performs a backward analysis and identifies the slice of instructions that influence R1. The register R1 is written at P8; hence P8 belongs to the slice. The branch at P7 determines whether P9 executes or not; hence P7 belongs to the slice and contributes to the path condition. The branch at P7 uses register R2; hence every instruction that produces R2 also belongs to the slice. In the process of this backward analysis, instructions P6, P5, P4, P3, P2, P1, and P0 belong to the slice. The process terminates when there is a call to an external function (*i.e.*, helper function).

Next in the forward analysis, the kernel constructs a symbolic expression. There is only one leaf node (*i.e.*, register R0) from the backward analysis. Hence, it gets a symbolic value (*i.e.*,  $sym$ ). In the forward analysis, the verifier creates a symbolic expression for the access at P9 using the symbolic value. At P2, the register R1 is assigned the frame pointer R10 and R10 is assigned a buffer of 16 bytes on the stack after P3 and it is at the base of the buffer. The symbolic expression for R1 after P8 is  $(sym \ \& \ 15) + 4$ . The read of R1 at P9 is unsafe if it is greater than 15. Hence, the symbolic expression for the access to be unsafe is  $(sym \ \& \ 15) + 4 > 15$ . The symbolic value for the path condition  $R2 \geq 4$  is  $15 - (sym \ \& \ 15) \geq 4$ . The refinement condition generated by the kernel verifier is  $(15 - (sym \ \& \ 15) \geq 4) \wedge ((sym \ \& \ 15) + 4 > 15)$ . If the refinement condition is unsatisfiable, then the access at P9 is safe.

#### Discharging the refinement condition to userspace.

Rather than using an SMT solver within the kernel, the BCF framework delegates checking the refinement condition to userspace. The kernel saves the verifier state (by pinning it to an anonymous file descriptor that can be used to resume

| step | rule      | what it establishes                                                                                                                                                    | derived from   |
|------|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| 0    | ASSUME    | $(15 - (sym \ \& \ 15) \geq 4) \wedge ((sym \ \& \ 15) + 4 > 15)$                                                                                                      | refinement     |
| 1    | AND_ELIM  | $15 - (sym \ \& \ 15) \geq 4$                                                                                                                                          | step 0         |
| 2    | AND_ELIM  | $(sym \ \& \ 15) + 4 > 15$                                                                                                                                             | step 0         |
| 3    | BIT_BLAZT | $(sym \ \& \ 15) \leq 15$                                                                                                                                              |                |
| 4    | POLY_NORM | $(sym \ \& \ 15) + 4 > 15$ from step 2,<br>$(sym \ \& \ 15) \leq 15$ from step 3<br>$\Rightarrow (sym \ \& \ 15) \geq 12$<br>$\Rightarrow 15 - (sym \ \& \ 15) \leq 3$ | step 2, step 3 |
| 5    | CONTRA    | $15 - (sym \ \& \ 15) \leq 3$ from step 4<br>$15 - (sym \ \& \ 15) \geq 4$ from step 1<br>$\Rightarrow \text{False}$                                                   | step 4, step 1 |

**Figure 3.** The proof generated by BCF’s untrusted CVC5 solver. Given the refinement condition, the AND\_ELIM rule splits the assumed conjunction into constituents. The BIT\_BLAZT rule reasons about the individual bits. The POLY\_NORM rule states these two terms are equal because they have the same polynomial normal form. The CONTRA rule proves the refinement by deriving False.

the verifier) and returns from the bpf syscall with an error (see Step 3 in Figure 2) along with the refinement condition.

**Proof object from userspace SMT solver.** BCF uses an untrusted SMT solver in userspace to prove the unsatisfiability of the refinement condition. If it can prove that the refinement condition is UNSAT, then BCF’s CVC5 solver also emits a proof. The proof object generated is structured as a set of rules to derive a contradiction for the refinement condition. Figure 3 shows the proof to derive a contradiction with the refinement condition that the kernel wanted to prove. Then this proof object is passed to the kernel verifier with another bpf syscall (see Step 5 in Figure 2).

**In-kernel proof checker.** The kernel verifier resumes verification by restoring the verifier state from the anonymous file descriptor. It will resume verification from P9. It first checks the proof with an in-kernel proof checker. Importantly, the kernel only trusts this proof checker. If the proof is deemed to be valid, it continues verification after tightening register R1’s interval bounds.

**Multiple refinement steps.** For the example in Figure 1, the kernel verifier will trigger the refinement procedure at P9 and P12. Hence, the steps 2-3-4-5-6 in Figure 2 need to be repeated for the refinement step at P12.

**Limitations of BCF.** The BCF framework performs multiple userspace-to-kernel transitions for each refinement step. In a large program, it may require multiple such refinement steps. These steps have multiple drawbacks. They increase the latency of loading a bpf program. They require invoking and waiting on the result from an untrusted SMT solver in a production kernel, which complicates the path to deployment. The verifier state is held in kernel memory for an unbounded interval of time controlled by userspace behind an anonymous file descriptor. For a large program where the kernel verifier could reject the program at  $n$  steps, it requires  $n + 1$  userspace-to-kernel syscalls. We address these limitations with our proof-carrying-BPF framework.

### 3 Proof-Carrying-BPF Programs

We describe our **Proof-Carrying-BPF (PCB)** framework that increases the class of programs accepted by the kernel verifier while keeping the trusted computing base small. Our key idea is that the BPF backend should not only generate bytecode but also refinement proofs at all points where the kernel verifier would fail, which is inspired by prior work on proof-carrying code [15, 16].

To identify the potential places where the kernel verifier would fail, we create a userspace version of the kernel verifier and couple it with the BPF backend. We present a realization of our PCB framework by reusing the in-kernel proof checker of BCF that we described in Figure 3. In contrast to BCF [25], our PCB framework delegates the task of producing proofs to the BPF backend and eliminates the userspace-kernel transitions and complexity in the kernel to suspend/save/restore verifier state. Our PCB framework has the added benefit of removing the “fighting-the-verifier” frustration of developers because any code that is generated by our BPF backend is accepted by the kernel verifier.

**The PCB framework.** Figure 4 illustrates the components and the workflow of our PCB framework. It has two main components: (1) a backend with a userspace verifier that compiles the BPF program and generates the proofs and (2) the modifications to the kernel verifier that load the BPF program along with proofs, generate the symbolic expressions, and check the proofs during refinement.

**Our userspace verifier.** Our BPF backend has a userspace verifier attached to the typical clang compiler, which analyzes the generated bytecode and generates the output only when the kernel verifier can successfully verify it either with the default abstract domains or with the refinement procedure. The userspace verifier is a mirror of the kernel verifier and uses the same default abstract domains. Hence, wherever the program passes the default analysis, no further proofs are needed.

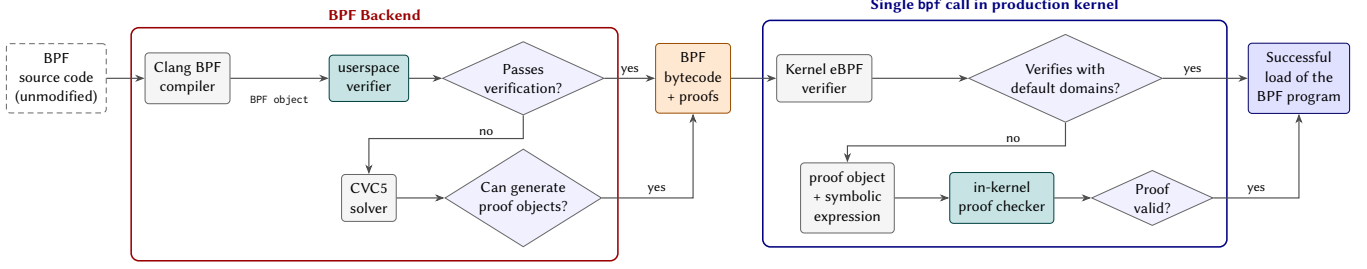
When the userspace verifier identifies a program point where the kernel’s non-relational domains cannot prove safety, the userspace verifier produces a refinement condition (see Section 2) and solves it using the CVC5 solver. If the SMT solver ascertains that the refinement condition is UNSAT, then the access is safe. The userspace verifier generates the proof object (see Figure 3) and emits the proof object for the program point. The userspace verifier stores the refinement condition and the corresponding proof object in a hash table indexed by the hash of the serialization of the refinement condition. Our BPF backend’s userspace verifier mirrors the kernel’s verifier, so both the backend and the kernel produce the same hash when they are checking the safety of the same refinement condition. The backend generates all the refinement conditions and the corresponding proof objects along with the BPF bytecode in a single pass when the bytecode is being generated.

Another kind of refinement happens when the access whose safety is being checked lies on an infeasible path. The userspace verifier identifies it using the infeasibility of the path condition. Our userspace verifier conveys the infeasibility of such paths using the proof object. Note that this capability is not present in the stock kernel verifier, which conservatively explores such paths and rejects the program; proving a path infeasible became possible with the refinement interface that BCF introduced, and PCB precomputes these infeasibility proofs like any other refinement.

**Kernel verifier.** The kernel verifier loads the BPF bytecode and the bundled proof objects in a single bpf syscall with the BPF\_PROG\_LOAD command (right side of Figure 4). The kernel verifier runs as it does under BCF, with one difference: when it reaches a check that does not successfully verify with its default domains, rather than suspending and round-tripping to user space, it reconstructs the refinement condition for that check and computes its canonical hash. It uses the hash to find the matching entry in the bundle of proof objects. It confirms that the entry’s refinement condition is exactly the same as the one it constructed. If so, it validates the proof with the in-kernel checker before continuing. If the proof is valid, it continues verification of the program. This design completely eliminates all load-time userspace-kernel transitions.

**Implementation challenges.** One of the key challenges in implementing the userspace verifier is maintaining fidelity with the kernel verifier. We developed it in an incremental fashion by building a skeleton that mirrors the kernel verifier’s structure (an instruction-by-instruction abstract interpreter over per-program-point states). We used the kernel’s selftest suite as an executable specification of verifier behavior in this test-driven approach. Whenever there is a divergence in the behavior between the kernel and userspace verifiers, we compare our logic against the corresponding kernel function and port the kernel’s exact rule. Among the selftests, divergences with false accepts (we accept, the kernel rejects) are soundness bugs and are fixed first; false rejects only cost coverage and are handled subsequently. In this methodology, handling the abstract domains is the easier task. In contrast, the kernel’s pruning logic is harder: the kernel explores every path depth-first and, to prevent path explosion, compares the current abstract state against previously explored states at designated prune points, stopping if an earlier state subsumes the current one. Because our proofs must attach to the rejection points the kernel actually reaches, the mirror must prune *exactly* where the kernel prunes. This entails reproducing each ingredient of that decision: the same exploration order, the same per-register and per-stack-slot subsumption checks gated by the same liveness analysis, the same precision analysis that records which registers actually influenced a safety decision, and the kernel’s loop bookkeeping. Our verifier converged in stages,





**Figure 4.** Workflow of the Proof-Carrying-BPF framework where the backend has a mirrored userspace verifier that generates proofs for all the program points where the verification will fail with the default abstract domains and generates BPF bytecode with proofs, which is subsequently checked by the in-kernel proof checker during the bpf syscall.

from approximate to kernel-faithful as the selftests and real-world programs (Section 4) exposed remaining divergences.

**Summary.** Our PCB framework increases the class of programs accepted by the verifier. The modifications to the kernel verifier are significantly smaller than those in BCF because there is no need to save/restore verifier state or perform userspace-kernel transitions. Finally, the userspace verifier can serve as a good playground to explore richer verification strategies and various proof checkers.

## 4 Experimental Evaluation

**Setup.** We reuse the suite of 360 real-world eBPF programs assembled and publicly released by BCF [25]. This suite is built using production eBPF source from projects such as the Cilium and Calico CNI datapaths, the Inspektor Gadget (IG) tracing tools, and BCC. Each program is compiled across a range of compiler versions and optimization levels (clang-13 through clang-21 at -O1 to -O3) to obtain diverse BPF bytecode patterns. The suite contains both CO-RE and non-CO-RE builds: PCB parses the .BTF.ext section and applies the CO-RE relocations against the target kernel’s BTF before verification, and 177 of the 360 objects are CO-RE variants. All programs in the suite are rejected by the default kernel verifier. Our PCB prototype’s userspace verifier is written in Rust and attached to clang’s BPF backend through a shim wrapper. We modified the kernel verifier from BCF to remove the suspend/restore mechanisms and add the processing of bundles of refinement conditions and proof objects using the hashes of the refinement conditions. Our PCB prototype is open-source [4].

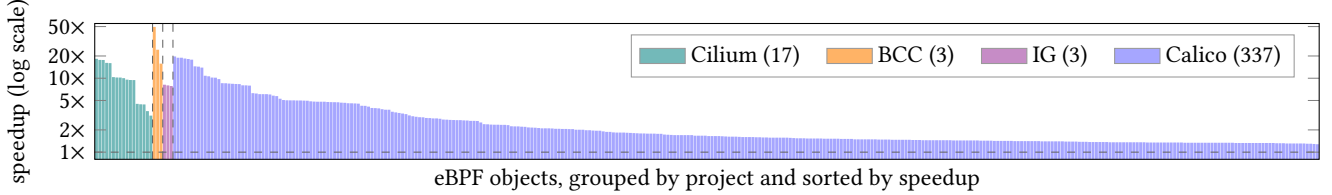
**Methodology.** We compare our PCB prototype against the BCF framework [25]. Both systems are evaluated on the same machine, a CloudLab c220g2 node with two 10-core Intel Xeon E5-2660 v3 CPUs at 2.60 GHz (40 hardware threads) and 160 GB of RAM, running Ubuntu 22.04 (Linux 5.15). Every program is loaded inside a QEMU/KVM guest configured identically for both systems (16 vCPUs and 16 GB of RAM): the guest boots our BCF-patched kernel for PCB and the kernel image published by the BCF authors for BCF. We refer to the whole BPF object being loaded and fully

accepted by the kernel as the load in the rest of this section (the single bpf system call on the right half of Figure 4). We use the kernel’s BPF selftest suite to test the faithfulness of the verifier. The evaluation target is the set of 360 BPF objects from the BCF suite that BCF accepts (337 Calico, 17 Cilium, 3 BCC, and 3 IG objects); our prototype generates bytecode and proof bundles for all of them. The load time is reported as the median of five runs following BCF’s convention. We refer to the time taken (one-time cost) for userspace verification with the BPF backend to produce the bytecode along with the bundle of refinement conditions and proof objects as the build time (the backend on the left half of Figure 4).

**Is our backend faithful to the kernel verifier?** We want to evaluate whether our BPF backend with the userspace verifier reaches the same accept or reject verdict as the kernel on every program. We measure this with the kernel’s BPF selftest suite [13], a corpus of hand-crafted tests targeting the verifier. Across 3,267 selftests, our backend agrees with the kernel verifier’s behavior in both directions: it accepts every test the kernel accepts and rejects every test the kernel rejects (zero false accepts and zero false rejects).

**Does PCB increase the class of programs accepted?** We want to evaluate whether our PCB framework allows programs that the kernel verifier would otherwise reject to be accepted with meaningful proofs. PCB allows all 360 production objects in the target suite to be accepted with our proof bundles: the entire Cilium TC datapath (17 objects), all 337 Calico objects, the BCC ksnop tool (3 objects), and the IG seccomp tools (3 objects). PCB also does not shrink the class of accepted programs: a program that is already accepted by the kernel needs no proofs and its load is unchanged.

**Does the PCB framework result in better load times when compared to BCF?** We want to evaluate whether using the userspace verifier in the backend that eliminates kernel-to-userspace transitions helps in improving load times. Figure 5 reports the speedup in load time with our PCB framework in comparison to BCF for every object in the corpus. PCB loads faster than BCF on *all 360 objects*: the speedup ranges from 1.28× to 49.5× with a median of 1.66× and an average of 3.4×. The speedup is significant when BCF



**Figure 5.** Per-object load-time speedup of PCB over BCF across all 360 objects of the evaluation corpus (median of five runs per object on both systems, same machine). Each bar is one object; within each project, objects are sorted by decreasing speedup. PCB loads faster on every object; the dashed line marks parity (1 $\times$ ).

performs multiple interactive refinement procedures: the median speedup is 1.6 $\times$  on Calico (up to 19.7 $\times$ ), 10 $\times$  on Cilium, 8 $\times$  on IG, and 24 $\times$  on BCC. The overall speedup depends on the number of kernel-to-userspace transitions in BCF. Across the 360 objects, BCF performs 40,672 kernel-to-userspace round trips to prove the refinement conditions at load time whereas PCB performs zero because they are precomputed by the backend. Loading the whole corpus once takes 2,181 seconds with PCB versus 6,886 seconds with BCF.

**Refinement condition and proof object sizes.** To reduce the load time, our backend precomputes the refinement conditions and proof objects as a bundle. It does increase the build time but it is done offline, not when the kernel loads the program. The backend produces one bundle that is the aggregation of all the refinement conditions and the proof objects for all the program points where the kernel verifier will invoke the refinement procedure. Across the corpus the bundle sizes range from 15 KB to 59 MB with a median of 0.6 MB, and 90% of the bundles are under 12 MB. The largest bundles belong to the BCC ksnoop objects, whose variable-length copies require many range-refinement obligations; these are also the objects with the largest load-time speedups.

## 5 Discussion

We discuss some challenges and potential solutions for deployment in a production kernel.

**Cost of program changes.** A proof bundle is tied to a program and the specific verifier implementation in the kernel. If either changes, the kernel’s refinement conditions may no longer match the corresponding versions in the proof bundle. In such cases, the kernel will reject the program under PCB. Because the kernel independently re-checks every proof, neither a stale proof bundle nor a divergent userspace mirror can compromise soundness. Recovering from staleness means re-running the build, a one-time cost of about a minute for the median object that can be automated alongside kernel updates. The same fail-closed property extends to the userspace mirror itself: if the kernel verifier evolves and the mirror falls behind, the cost is coverage, not safety. Re-running the kernel’s selftest suite takes minutes and identifies such mismatches, which can then be used to port the changes from the kernel to the userspace verifier.

**Proof bundle reuse and caching.** The checks that trigger refinement are often located in the same set of functions across programs because production programs are built with shared libraries. The primary cause of refinement often lies in such library code rather than in other parts unique to each program. Among the 17 Cilium objects, those built by the same compiler version (bpf\_host, bpf\_lxc, bpf\_wireguard, and bpf\_overlay) produce byte-identical bundles because all of their proof obligations arise in the shared datapath library. Similarly, some adjacent compiler versions (clang-14/15 and clang-18/19) also produce identical bundles. We can cache these bundles, which we call *bundle caching*, to store the built bundles and reuse them on later loads instead of building them again. Caching reduces the build time, which is the cost PCB pays for precomputing proofs. Because a bundle is valid on every host running the same kernel, this cost is paid once per distinct bundle rather than once per object, host, or load.

**Dynamically generated programs.** Caching does not help programs that are generated or patched at deployment time and loaded only once: each variant needs its own build, and the cost is paid in full for a single load. For such programs, BCF-style on-demand proving is likely better. The load of the BPF object itself remains faster with PCB, which also keeps the solver out of the kernel’s load path, but the total latency advantage is BCF’s. Whether these variants truly require separate builds is an open question. Cilium, for example, generates a per-node variant of its datapath at deployment time by substituting node-specific values, such as addresses and endpoint identities, into the constants of a program template. If the refinement conditions are stable under this constant patching, a single bundle would serve all the per-node variants; we have not explored it yet.

## 6 Related Work

**Proof-carrying code.** We are inspired by the extensive work on proof-carrying code for kernel extensions [15, 16]. The kernel publishes a safety policy together with a verification-condition generator. The untrusted code producer ships native code accompanied by a formal proof that the code satisfies the policy, which the kernel validates before executing the code without any run-time checks. Classic proof-carrying

code proves that the *whole* program satisfies the policy. Our PCB framework applies the same principle to the eBPF setting, but leaves the kernel’s existing verifier to discharge the parts it can already prove and carries proofs *only* for the program points where its abstract domains would otherwise reject the program.

In LPC 2021, Nelson *et al.* [17] explored a proof-carrying approach. They reduced the kernel’s role to checking a userspace-supplied proof against a specification. Applications generate the proofs, and the kernel only validates them and ensures that a valid proof cannot be fabricated. Their prototype, ExoBPF, expresses the specification, the proofs, and the checker in the Lean theorem prover, and provides two userspace proof generators, an abstract interpreter that mimics the current verifier and a symbolic-execution engine backed by a SAT solver. The difference between PCB and ExoBPF is that our approach requires significantly fewer changes to the kernel. While ExoBPF minimizes the kernel by replacing the verifier’s full analysis with proof checking against a Lean specification, PCB leaves the kernel verifier in place and adds proofs only at the points it would reject, reusing BCF’s in-kernel checker. ExoBPF is a very early-stage prototype and cannot run production-ready programs yet.

**The BCF framework.** The closest related work is the BCF framework [25], which also couples the kernel verifier with an untrusted user-space solver and a small in-kernel proof checker. The two approaches differ in *when* and *how* the proofs are produced. BCF discharges each rejected check on demand: when the verifier reaches a point it cannot prove safe, it suspends, transitions to user space for a proof, checks the proof, and resumes. This is repeated for every such point. In contrast, PCB analyzes the whole program ahead of time in a bpf backend that has a userspace mirror of the kernel verifier and aggregates the proofs for all of these points into a single bundle, which improves load time.

**Userspace verification with PREVAIL.** The best-known userspace verifier is PREVAIL [7], which serves as the verifier for eBPF on Windows. Like the kernel verifier, PREVAIL is an abstract interpreter, but it makes two choices that allow it to accept more programs. First, PREVAIL uses a *relational* zone abstract domain that also tracks arithmetic relationships between registers. Tracking these relationships helps PREVAIL reject fewer safe programs. Second, rather than exploring program paths one at a time, PREVAIL computes a fix-point over the control-flow graph, which enables it to handle loops. In contrast to PCB, adopting PREVAIL in Linux would require either swapping the in-kernel verifier for a larger analyzer, or trusting the verdict of an external tool.

**Improving the eBPF verifier.** A large body of work has made the kernel verifier precise and improved its soundness [2, 8, 9, 22, 23, 28–30]. However, these efforts are primarily constrained by the limits of what can be accepted by the kernel’s abstract domains. In contrast, PCB extends the class of programs substantially more than these prior efforts.

**Safe kernel extensions.** Safely extending the kernel is a classic problem that predates eBPF. The SPIN operating system [1] accepts extensions written in the type-safe language Modula-3 and trusts its compiler to confine code dynamically linked into the kernel address space. VINO [20] instead confines extensions with software fault isolation [31], which rewrites the extension’s object code to sandbox its memory accesses, and wraps extensions in lightweight transactions to undo the effects of misbehaving code. Both systems either trust a compiler or resort to run-time checks, whereas the eBPF verifier establishes safety statically at load time.

A different line of work avoids the verifier by changing the language in which eBPF extensions are written. REX [10] lets developers write extensions in safe Rust, closing a language-verifier gap in which safe programs are rejected because the verifier’s notion of safety differs from the one the source language already guarantees. The Rust compiler enforces the static safety properties and emits native code directly, and a lightweight runtime handles the properties that resist static checking, such as safe exception handling, stack safety, and termination. This removes the verifier altogether, but it shifts trust to the Rust toolchain and its runtime. PCB, by contrast, retains the kernel verifier as the final judge and the eBPF program as the unit of loading, and widens the class of accepted programs by supplying proofs rather than by replacing the language or the verifier.

## 7 Conclusion

Our Proof-Carrying-BPF framework widens the class of programs the kernel accepts without greatly increasing its trusted computing base. PCB pairs the BPF backend with a userspace mirror of the kernel verifier. The mirror predicts the program points that the kernel cannot prove safe with its default abstract domains, proves the refinement conditions with an SMT solver, and collects the resulting proofs into a bundle that ships with the program. The kernel then loads the program in a single pass, running its own verifier and consulting the proof bundle only where its analysis falls short, validating each proof with a small in-kernel checker. PCB builds on the BCF framework but removes its interactive refinement loop, so verification no longer suspends, pins kernel state, or round-trips to user space during a load. PCB not only successfully verifies previously rejected programs but also loads them faster. We believe that PCB is a practical way to let the kernel accept more safe programs efficiently while keeping the verifier extensions small.

## Acknowledgments

This paper is based upon work supported in part by the research gift from the eBPF Foundation and the National Science Foundation grant: NSF-2422076. We also thank Cloud-Lab for providing the testbed for our experiments.

## References

- [1] Brian N. Bershad, Stefan Savage, Przemyslaw Pardyak, Emin Gün Sirer, Marc E. Fiuczynski, David Becker, Craig Chambers, and Susan Eggers. 1995. Extensibility safety and performance in the SPIN operating system. In *Proceedings of the Fifteenth ACM Symposium on Operating Systems Principles*. Association for Computing Machinery, New York, NY, USA, 267–283. doi:10.1145/224056.224077
- [2] Sanjit Bhat and Hovav Shacham. 2022. Formal Verification of the Linux Kernel eBPF Verifier Range Analysis. <https://sanjit-bhat.github.io/assets/pdf/ebpf-verifier-range-analysis22.pdf>.
- [3] Daniel Borkmann. 2023. bpf: Fix \_\_reg\_bound\_offset 64->32 var\_off subreg propagation. <https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git/commit/?id=7be14c1c9030f73cc18b4ff23b78a0a081f16188>. Accessed: 2026-06-17.
- [4] Yalu Cai and Santosh Nagarakatte. 2026. Alivio. <https://github.com/rutgers-apl/Alivio>. Accessed: 2026-08-03.
- [5] John Fastabend. 2020. bpf: Verifier, adjust\_scalar\_min\_max\_vals to always call update\_reg\_bounds(). <https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git/commit/?id=294f2fc6da27620a506e6c050241655459ccd6bd>. Accessed: 2026-06-17.
- [6] Graeme Gange, Zequn Ma, Jorge A. Navas, Peter Schachte, Harald Søndergaard, and Peter J. Stuckey. 2021. A Fresh Look at Zones and Octagons. *ACM Trans. Program. Lang. Syst.* 43, 3, Article 11 (Sept. 2021), 51 pages. doi:10.1145/3457885
- [7] Elazar Gershuni, Nadav Amit, Arie Gurfinkel, Nina Narodytska, Jorge A. Navas, Noam Rinetzy, Leonid Ryzhyk, and Mooly Sagiv. 2019. Simple and precise static analysis of untrusted Linux kernel extensions. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Phoenix, AZ, USA) (PLDI 2019). Association for Computing Machinery, New York, NY, USA, 1069–1084. doi:10.1145/3314221.3314590
- [8] Google. 2023. Buzzer - An eBPF Fuzzer toolchain. <https://github.com/google/buzzer>. Accessed: 2026-06-17.
- [9] Hsin-Wei Hung and Ardalan Amiri Sani. 2024. BRF: Fuzzing the eBPF Runtime. *Proc. ACM Softw. Eng.* 1, FSE, Article 52 (July 2024), 20 pages. doi:10.1145/3643778
- [10] Jinghao Jia, Ruowen Qin, Milo Craun, Egor Lukyanov, Ayush Bansal, Minh Phan, Michael V. Le, Hubertus Franke, Hani Jamjoom, Tianyin Xu, and Dan Williams. 2025. Rex: Closing the Language-Verifier Gap with Safe and Usable Kernel Extensions. In *Proceedings of the 2025 USENIX Conference on Usenix Annual Technical Conference* (USA, 2025-07-07) (USENIX ATC '25). USENIX Association, 325–342.
- [11] Juan Jose. 2023. Linux Kernel: Vulnerability in the eBPF verifier register limit tracking. <https://github.com/google/security-research/security/advisories/GHSA-hfqc-63c7-rj9f>. Accessed: 2026-06-17.
- [12] Lucas Leong. 2021. CVE-2021-31440: An Incorrect Bounds Calculation in the Linux Kernel eBPF Verifier. <https://www.zerodayinitiative.com/blog/2021/5/26/cve-2021-31440-an-incorrect-bounds-calculation-in-the-linux-kernel-ebpf-verifier>. Accessed: 2026-06-17.
- [13] Linux Kernel Community. 2024. eBPF In-Kernel Test Suite (self-tests/bpf). <https://elixir.bootlin.com/linux/v6.14/source/tools/testing/selftests/bpf>. Accessed: 2026-06-17.
- [14] Andrii Nakryiko. 2023. bpf register bounds logic and testing improvements. <https://git.kernel.org/pub/scm/linux/kernel/git/bpf/bpf-next.git/commit/?id=cd9c127069c040d6b022f1ff32fed4b52b9a4017>. Accessed: 2026-06-17.
- [15] George C. Necula. 1997. Proof-carrying code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Paris, France) (POPL '97). Association for Computing Machinery, New York, NY, USA, 106–119. doi:10.1145/263699.263712
- [16] George C. Necula and Peter Lee. 1996. Safe kernel extensions without run-time checking. In *Proceedings of the Second USENIX Symposium on Operating Systems Design and Implementation* (Seattle, Washington, USA) (OSDI '96). Association for Computing Machinery, New York, NY, USA, 229–243. doi:10.1145/238721.238781
- [17] Luke Nelson, Xi Wang, and Emina Torlak. 2021. A Proof-Carrying Approach to Building Correct and Flexible In-Kernel Verifiers. Linux Plumbers Conference (LPC). Accessed: 2026-06-17. <https://lpc.events/event/11/contributions/944/>.
- [18] Valentina Palmiotti. 2021. Kernel pwning with eBPF: a love story. <https://chomp.ie/Blog+Posts/Kernel+Pwning+with+eBPF++a+Love+Story>. Accessed: 2026-06-17.
- [19] Manfred Paul. 2020. CVE-2020-8835: Linux kernel privilege escalation via improper eBPF program verification. <https://www.zerodayinitiative.com/blog/2020/4/8/cve-2020-8835-linux-kernel-privilege-escalation-via-improper-ebpf-program-verification>. Accessed: 2026-06-17.
- [20] Margo I. Seltzer, Yasuhiro Endo, Christopher Small, and Keith A. Smith. 1996. Dealing with disaster: surviving misbehaved kernel extensions. In *Proceedings of the Second USENIX Symposium on Operating Systems Design and Implementation*. Association for Computing Machinery, New York, NY, USA, 213–227. doi:10.1145/238721.238779
- [21] Matan Shachnai. 2024. bpf, verifier: Improve precision of BPF\_MUL. <https://git.kernel.org/pub/scm/linux/kernel/git/bpf/bpf-next.git/commit/?id=9aa0ebde0014>. Accessed: 2026-06-17.
- [22] Matan Shachnai, Harishankar Vishwanathan, Srinivas Narayana, and Santosh Nagarakatte. 2024. Fixing Latent Unsound Abstract Operators in the eBPF Verifier of the Linux Kernel. In *Static Analysis*. Springer Nature Switzerland, 386–406.
- [23] Matan Shachnai, Harishankar Vishwanathan, Srinivas Narayana, and Santosh Nagarakatte. 2025. Comparing the Precision of Abstract Operators in the eBPF Verifier Using Differential Synthesis. In *Static Analysis: 32nd International Symposium, SAS 2025, Singapore, Singapore, October 13–14, 2025, Proceedings* (Singapore, Singapore). Springer-Verlag, Berlin, Heidelberg, 142–166. doi:10.1007/978-3-032-07106-4\_7
- [24] Hao Sun and Zhendong Su. 2024. Validating the eBPF Verifier via State Embedding. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, 615–628. <https://www.usenix.org/conference/osdi24/presentation/sun-hao>
- [25] Hao Sun and Zhendong Su. 2025. Prove It to the Kernel: Precise Extension Analysis via Proof-Guided Abstraction Refinement. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles* (Lotte Hotel World, Seoul, Republic of Korea) (SOSP '25). Association for Computing Machinery, New York, NY, USA, 736–751. doi:10.1145/3731569.3764796
- [26] Hao Sun, Yiru Xu, Jianzhong Liu, Yuheng Shen, Nan Guan, and Yu Jiang. 2024. Finding Correctness Bugs in eBPF Verifier with Structured and Sanitized Program. In *Proceedings of the Nineteenth European Conference on Computer Systems* (Athens, Greece) (EuroSys '24). Association for Computing Machinery, New York, NY, USA, 689–703. doi:10.1145/3627703.3629562
- [27] Harishankar Vishwanathan. 2025. bpf, verifier: Improve precision for BPF\_ADD and BPF\_SUB. <https://git.kernel.org/pub/scm/linux/kernel/git/bpf/bpf-next.git/commit/?id=7a998a731627>. Accessed: 2026-06-17.
- [28] Harishankar Vishwanathan, Matan Shachnai, Srinivas Narayana, and Santosh Nagarakatte. 2022. Sound, Precise, and Fast Abstract Interpretation with Tristate Numbers. In *Proceedings of the 20th IEEE/ACM International Symposium on Code Generation and Optimization* (Virtual Event, Republic of Korea) (CGO '22). IEEE Press, 254–265. doi:10.1109/CGO53902.2022.9741267
- [29] Harishankar Vishwanathan, Matan Shachnai, Srinivas Narayana, and Santosh Nagarakatte. 2023. Verifying the Verifier: eBPF Range Analysis Verification. In *Computer Aided Verification: 35th International*



- Conference, CAV 2023, Paris, France, July 17–22, 2023, Proceedings, Part III* (Paris, France). Springer-Verlag, Berlin, Heidelberg, 226–251. doi:10.1007/978-3-031-37709-9\_12
- [30] Harishankar Vishwanathan, Matan Shachnai, Srinivas Narayana, and Santosh Nagarakatte. 2025. Automatic Synthesis of Abstract Operators for eBPF. In *Proceedings of the 3rd Workshop on EBPF and Kernel Extensions* (Coimbra, Portugal) (*eBPF '25*). Association for Computing Machinery, New York, NY, USA, 15–21. doi:10.1145/3748355.3748361
- [31] Robert Wahbe, Steven Lucco, Thomas E. Anderson, and Susan L. Graham. 1993. Efficient software-based fault isolation. In *Proceedings of the Fourteenth ACM Symposium on Operating Systems Principles*. Association for Computing Machinery, New York, NY, USA, 203–216. doi:10.1145/168619.168635
- [32] Shung-Hsi Yu. 2024. bpf, verifier: improve signed ranges reasoning. <https://lore.kernel.org/bpf/20240719110059.797546-6-xukuohai@huaweicloud.com/>. Accessed: 2026-06-17.